

1. Project Title: Real-Time Taxi Ride Analytics Pipeline

2. Type of System: Streaming Data Engineering Pipeline

-> The project simulates how a company processes real-time transportation data and generates analytics such as:

- ride counts
- average fares
- busiest pickup zones
- ride demand trends

Instead of processing data in batches, the system processes taxi ride events continuously as they arrive.

3. Logical Problem Statement:

-> Many modern systems (ride-sharing apps, financial transactions, IoT devices) generate continuous streams of data.

-> Traditional batch systems process data after it is stored, which causes delays.

-> Goal of the project:- Build a pipeline that can:

- ingest streaming data
- process it in real time
- simulate real-world streaming systems

-> Example use case: Taxi companies want to analyze ride data as it happens to monitor:

- ride demand
- location trends
- trip duration
- revenue patterns

4. Project Problem Statement:

-> Urban transportation systems generate massive streams of ride data every second.

-> Traditional batch processing systems analyze data hours later, which is not suitable for real-time decision making.

-> Example problems:

- city authorities cannot monitor ride demand in real time
- taxi companies cannot detect busy zones immediately
- analytics dashboards update too slowly.

-> Therefore, we need a real-time data pipeline that can:

- ingest streaming ride events
- process them instantly
- store metrics for analysis
- enable real-time insights

5. Aim of the Project

The aim is to design and implement a real-time data engineering pipeline that:

1. Simulates taxi ride events from the NYC Taxi dataset
2. Streams the data using Kafka
3. Processes the stream using Spark
4. Stores analytics metrics in Cassandra
5. Uses Airflow to orchestrate and manage the pipeline

This project demonstrates core data engineering concepts:

- streaming ingestion
- distributed processing
- scalable storage
- workflow orchestration

6. Dataset:

-> The data is taken from the **NYC Taxi dataset** from the New York City Taxi and Limousine Commission.

-> The dataset contains:

- pickup location
- dropoff location
- passenger count
- trip distance
- fare amount
- Timestamps

We simulate this historical dataset as live streaming events. Instead of sending the whole dataset at once, we send rows one by one with a delay.

7. Project Overview:

-> This project builds a real-time data engineering pipeline that processes taxi ride events and produces analytics that can be visualized on a dashboard. The dataset used is the NYC Taxi Trip dataset, which contains information about taxi rides such as pickup time, drop-off time, distance, fare amount, etc. Instead of processing the dataset as a static batch, the project simulates real-time streaming of taxi rides and processes them continuously to generate insights.

8. Goal of the Project:

-> The aim is to demonstrate how modern data engineering systems handle streaming data pipelines.

-> The pipeline will:

1. Ingest ride events
2. Stream them through a messaging system
3. Process them in real time
4. Store analytics metrics
5. Visualize insights on a dashboard

9. Industrial tools being used:

- Python
- Apache Kafka
- Apache Spark
- Apache Cassandra
- Apache Airflow
- Docker
- Grafana/Power BI

10. Architecture of the Pipeline:

-> The system follows this architecture:



11. Explanation of Each Stage:

1. Dataset

- Source data containing historical taxi ride records.

2. Python Producer

- Reads rows from the dataset.
- Converts them to JSON messages.
- Sends them to Kafka one event at a time to simulate real-time streaming.

3. Python: Writing pipeline logic

- Simulate taxi ride events
- Send data to kafka
- Write spark streaming jobs
- Interact with Cassandra
- Write Airflow DAGs
- Used because it works good with spark, karka, airflow

4. Kafka: Real-time streaming platform / real time data ingestion

- Kafka acts as a distributed event streaming platform.
- Acts as a message broker (data pipeline highway)
- Stores incoming ride events in a topic called taxi_rides.
- Instead of sending data directly to Spark, we push it into Kafka first.
- Kafka stores data as topics. Every taxi ride becomes a message.
- Allows downstream systems to consume the stream.

- Used as it is the industry standard of event pipelines, handles high throughput streaming, decouples producers and consumers, scalable and fault-tolerant, also used by netflix and uber.

5. Zookeeper: Kafka cluster coordination

- ZooKeeper was used in Kafka to manage broker metadata, leader election, cluster coordination, and topic configuration. Modern Kafka versions are moving to KRaft mode to remove the ZooKeeper dependency.
- ZooKeeper manages broker metadata, leader election, cluster synchronization

6. Spark Streaming: Real time data processing

- Spark Structured Streaming reads events from Kafka and processes them in micro-batches.
- Read messages from Kafka in real time.
- Processes the ride events.
- Computes analytics metrics such as trip counts, averages, or revenue trends.
- Supports large scale data processing, structured streaming, and machine learning

7. Data Lakehouse Architecture : Bronze–Silver–Gold Layers

Bronze, Silver, and Gold layers represent different levels of data processing in modern data engineering systems.

-> Bronze Layer – Raw Data

- Stores raw taxi ride events exactly as received from Kafka.
- No transformation is applied.
- Acts as the raw historical record of the data stream.

->Silver Layer – Cleaned Data

- Raw events are cleaned and validated.
- Removes null values, invalid distances, or corrupt records.
- Converts timestamps into structured formats.

->Gold Layer – Aggregated Analytics

- Business-level analytics are generated.
- Example metrics:
 - average fare
 - average trip distance
 - ride counts
 - revenue trends
- These aggregated metrics are stored in Cassandra for fast querying.

This architecture is commonly used in modern data platforms such as Databricks Lakehouse architectures.

8. Cassandra: Data storage for analytics

- Storage layer for processed metrics
- Spark writes different levels of processed data into the system: Bronze Layer – raw ride events | Silver Layer – cleaned and validated ride data | Gold Layer – aggregated analytics metrics. Cassandra stores the final Gold layer analytics results so that they can be queried efficiently by dashboards such as Power BI or Grafana.
- Stores processed analytics results.

- Optimized for fast reads and writes for streaming workloads.
- Cassandra is used for High speed, Good for streaming data, and has a Distributed NoSQL database.

9. Apache Airflow: Pipeline Orchestration

- Airflow schedules and manages workflows.
- Instead of manually running scripts, Airflow runs them automatically.
- Airflow uses DAGs (Directed Acyclic Graphs).
- Airflow is used to automate pipelines, schedule jobs, monitor pipelines, and is the industry standard.

10. Docker: Infrastructure management

- Run all tools easily
- Installing Kafka, Spark, Cassandra manually is difficult.
- Docker lets us run them as containers.
- Benefits include easy steps, no dependency conflicts, same environment everywhere.

11. Power BI

- Connects to the processed data.
- Displays analytics through dashboards and visualizations.

STEP I

1. Made the project folder 'taxistream'
2. Inside the folder taxistream, made the folders:

```
>airflow
>data
>docker
>producer
    >producer.py
>spark
    >stream_processor.py
>docker-compose.yml
>Readme.md
>test_data.py
>requirements.txt
```

STEP II: ENVIRONMENT SETUP

1. In Anaconda Terminal we create a new environment:

```
>conda create -n taxic python=3.10
>conda activate taxic
```

2. We install Basic libraries:

```
>pip install kafka-python pyspark cassandra-driver pandas pyarrow apache-airflow
```

This will be used for Python Producer → Kafka, Spark Streaming, Cassandra connection.

- Pyarrow is used for .parquet files.
- kafka-python is used to connect Python producer to Kafka, and send taxi events to Kafka topics.
- pyspark is used to implement Spark streaming jobs, and process data from Kafka
- pandas is used to read and preprocess the NYC dataset, and simulate streaming data

- cassandra-driver is used to connect Spark/Python to Cassandra database, and insert processed analytics
 - apache-airflow is used to define workflow DAGs, and schedule pipeline tasks
3. We save the requirements in the taxistream folder as a txt file:
- ```
>conda list >requirements.txt
```

### STEP III: DATASET DOWNLOAD

1. Go to the link: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>  
(This dataset has yellow taxi trip records, green taxi trip records, for each month and year)
2. Download the "2025, yellow taxi trip records (parquet)" and put that inside >data folder.
3. So now the >data folder has >yellow\_tripdata\_2025-01.parquet.  
(.parquet is a columnar data format used in big data systems like Spark, hive, hadoop, so it actually makes the project look realistic)

.....

Testing:

1. We open the terminal:  

```
>activate taxic
>cd "C:\Users\rajve\Desktop\SkillIssues\Personal Projects\taxistream"
```
2. We made the >test\_data.py under the taxistream folder by using the terminal command:  

```
>type nul > test_data.py
```
3. Inside it, we write the code:  

```
import pandas as pd
df = pd.read_parquet("data/yellow_tripdata_2025-01.parquet")
print(df.head())
print("Total rows:", len(df))
```
4. We run this code in the terminal:  

```
>python test_data.py
```

```

VendorID tpep_pickup_datetime tpep_dropoff_datetime ... congestion_surcharge Airport_fee cbd_congestion_fee
0 1 2025-01-01 00:18:38 2025-01-01 00:26:59 ... 2.5 0.0 0.0
1 1 2025-01-01 00:32:40 2025-01-01 00:35:13 ... 2.5 0.0 0.0
2 1 2025-01-01 00:44:04 2025-01-01 00:46:01 ... 2.5 0.0 0.0
3 2 2025-01-01 00:14:27 2025-01-01 00:20:01 ... 0.0 0.0 0.0
4 2 2025-01-01 00:21:34 2025-01-01 00:25:06 ... 0.0 0.0 0.0

[5 rows x 20 columns]
Total rows: 3475226

```

5. Milestone completed: Dataset -> Python

### STEP IV: DOCKER INFRASTRUCTURE

(To set up docker containers for heavy tools such as: Zookeeper | kafka | Spark | Cassandra | Airflow. All these will be controlled by Docker Compose)

1. We installed Docker Desktop, Windows ARM64 from the link :

<https://www.docker.com/products/docker-desktop/> .

(Without Docker you would need to manually install and configure: Apache Kafka | Apache ZooKeeper | Apache Spark | Apache Cassandra | Apache Airflow. Each of those installations can take hours and break easily.

With Docker we run them using one command:

```
>docker compose up)
```

2. We can then check Docker version:

```
>docker --version
```

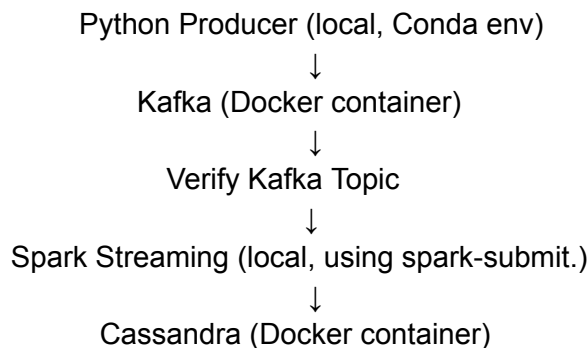
Docker version 20.10.1, build a5c7197

3. The >docker-compose.yml file will be used to start Apache Kafka and Apache Zookeeper inside docker containers. Hence, **Docker is the infrastructure layer** that runs Kafka, Spark, Cassandra, etc.

4. While the conda env manages python and its libraries(pandas, pyarrow, pyspark, kafka-spark), Docker on the other hand runs external services your Python code connects to, such as: Apache Kafka, Apache ZooKeeper, Apache Spark, Apache Cassandra, Apache Airflow.

### How they connect?:

Example pipeline:



**They communicate through ports like:** localhost:9092 → Kafka

So, in short, Python scripts use the tools to send and process data, while Docker runs the data infrastructure. Python and Docker work together using localhost connections.

5. After installing, we create the file >docker-compose.yml under the taxistream folder.

(This file will start the services your pipeline needs. For now we will start with Kafka infrastructure: Apache Kafka and Apache ZooKeeper.

Later we will add: Apache Cassandra, Apache Spark, Apache Airflow

6. In >docker-compose.yml, we write the code:

```
services:

 zookeeper:
 image: confluentinc/cp-zookeeper:7.5.0
 container_name: zookeeper
 environment:
 ZOOKEEPER_CLIENT_PORT: 2181
 ports:
 - "2181:2181"

 kafka:
```

```

image: confluentinc/cp-kafka:7.5.0
container_name: kafka
depends_on:
 - zookeeper
ports:
 - "9092:9092"
environment:
 KAFKA_BROKER_ID: 1
 KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
 KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
 KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

```

7. In the Anaconda prompt, we execute:

```
>docker compose up
```

It will ask for : View in Docker Desktop | View Config | Enable Watch | Detach

We execute:

```
>docker compose up -d
```

Docker will download images for Apache ZooKeeper and Apache Kafka (because the code has it only for zookeeper and kafka)

-> We can check the containers by:

```
>docker ps
```

```

(taxic) C:\Users\rajve\Desktop\SkillIssues\Personal Projects\taxistream>docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
fc372b32f9c9 confluentinc/cp-kafka:7.5.0 "/etc/confluent/dock... 3 days ago Up 2 days 0.0.0.0:9092->9092/tc
p, [::]:9092->9092/tcp kafka
1b42226a5712 confluentinc/cp-zookeeper:7.5.0 "/etc/confluent/dock... 3 days ago Up 2 days 0.0.0.0:2181->2181/tc
p, [::]:2181->2181/tcp zookeeper

```

So, Apache ZooKeeper is running on port 2181

Apache Kafka is running on port 9092

So our streaming infrastructure is now alive.

### STEP V: PYTHON KAFKA PRODUCER

(Python Kafka producer that sends taxi rides from the dataset into Kafka.)

1. We create file 'producer.py' under >producer folder in the taxistream folder.

2. In producer.py, we run the code:

```

from kafka import KafkaProducer
import pandas as pd
import json
import time

Kafka connection
producer = KafkaProducer(
 bootstrap_servers='localhost:9092',
 value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

```



```

Load dataset
df = pd.read_parquet('data/yellow_tripdata_2025-01.parquet')

Limit rows for testing
df = df.head(50)

topic = "taxi_rides"

for index, row in df.iterrows():

 data = row.to_dict()

 # Convert timestamps to string
 for key, value in data.items():
 if hasattr(value, "isoformat"):
 data[key] = value.isoformat()

 producer.send(topic, value=data)

 print(f"Sent ride {index}")

 time.sleep(0.5) # simulate real-time streaming

```

Then we run the command in anaconda prompt:

```
>python producer/producer.py
```

```

(taxic) C:\Users\rajve\Desktop\SkillIssues\Personal Projects\taxistream>python producer/producer.py
Sent ride 0
Sent ride 1
Sent ride 2
Sent ride 3
Sent ride 4
Sent ride 5
Sent ride 6
Sent ride 7
Sent ride 8
Sent ride 9
Sent ride 10
Sent ride 11
Sent ride 12
Sent ride 13
Sent ride 14
Sent ride 15

```

3. If for any reason, the keyline is : “NoBrokerAvailable”, that means that Kafka isn’t reachable at [localhost:9092](#) when the producer starts

What to do:

```
>docker compose down
```

```
>docker compose up -d ; and then,
>python producer/producer.py
```

Also, keep docker desktop on.

---

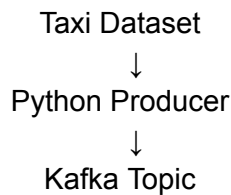
### BRIEF OF WHAT WE HAVE DONE TILL NOW:

1. We created a conda environment for the project, and installed libraries like: pandas, kafka-python, pyspark, and pyarrow. This environment is where our pipeline code runs.
2. **Infrastructure with Docker:** We installed Docker and started containers for: Apache ZooKeeper and Apache Kafka. These run as background services. Kafka is basically a real-time data stream platform.

#### Analogy:

Think of it like a message pipeline where systems can:  
send data → store temporarily → allow other systems to read it

3. Created a Kafka Producer: We wrote [producer.py](#), the program:
  - A. Reads the NYC taxi dataset
  - B. Converts each row to JSON
  - C. Sends it to Apache Kafka
4. Hence, so far, the flow is:



5. We also added: `time.sleep(0.5)`  
This **simulates real-time streaming** instead of sending everything instantly.

So our program sent:

Sent ride 0

Sent ride 1

...

Sent ride 49

These messages are now inside Kafka.

6. What Kafka is doing right now:

Kafka stores messages inside something called a **topic**.

Topic : taxi\_rides

So internally it looks like:

Kafka Topic: taxi\_rides

Ride 0

Ride 1

Ride 2

...

Kafka is waiting for another system to **consume** this stream.

## 7. Before moving forward, check whether kafka is working or not:

>docker exec -it kafka kafka-topics --list --bootstrap-server localhost:9092

```
(taxic) C:\Users\rajve\Desktop\SkillIssues\Personal Projects\taxistream>docker
exec -it kafka kafka-topics --list --bootstrap-server localhost:9092
taxi_rides
```

```
(taxic) C:\Users\rajve\Desktop\SkillIssues\Personal Projects\taxistream>
```

---

### BRIEF OF WHAT HAS TO BE DONE NOW:

#### 1. Need to create a consumer from Apache Spark, which will:

- Read Messages from Kafka
- Process them in real time
- Compute analytics metrics like: rides per minute, avg trip distance, avg fare amount, total revenue trends

2. Spark streaming consumer: Spark will continuously read the kafka stream, parse the incoming ride events, and perform real time processing. It will also compute metrics such as given above.

Cassandra will store analytics results, so that they can be queried efficiently.

Powerbi will visualize analytics such as: trip volume over time, revenue trends, avg trip details, peak ride hours.

3. Final Outcome: The completed system will function as a real-time taxi analytics platform, demonstrating how streaming data pipelines ingest, process, store, and visualize large volumes of event data.

---

### STEP VI: SPARK STREAMING SCRIPT

1. We create a file >stream\_processor.py under >spark folder in taxistream.

2. We write the code in stream\_processor.py:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json
from pyspark.sql.types import StructType, StringType, DoubleType

Create Spark session
spark = SparkSession.builder \
 .appName("TaxiStreamingProcessor") \
 .config("spark.sql.shuffle.partitions", "2") \
 .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

Schema for taxi data
schema = StructType() \
 .add("tpep_pickup_datetime", StringType()) \
 .add("tpep_dropoff_datetime", StringType()) \
```

```

 .add("trip_distance", DoubleType()) \
 .add("fare_amount", DoubleType())

Read stream from Kafka
df = spark.readStream \
 .format("kafka") \
 .option("kafka.bootstrap.servers", "localhost:9092") \
 .option("subscribe", "taxi_rides") \
 .option("startingOffsets", "latest") \
 .load()

Convert Kafka value to string
json_df = df.selectExpr("CAST(value AS STRING)")

Parse JSON
taxi_df = json_df.select(
 from_json(col("value"), schema).alias("data")
).select("data.*")

Compute analytics
analytics_df = taxi_df.groupBy().avg("fare_amount", "trip_distance")

Output to console (for testing)
query = analytics_df.writeStream \
 .outputMode("complete") \
 .format("console") \
 .start()

query.awaitTermination()

```

3. We go to <https://adoptium.net/> , and install Java from there. Temurin 17.0.18+8, Windows 64 bit (.MS

4. We see that this java file has been installed here:

C:\Program Files\Eclipse Adoptium\jdk-17.0.18.8-hotspot

Inside this, there should be a bin folder, having java.exe

#### **STEP VII: Setting JAVA\_HOME system variable**

1. Press Windows Key → Search “Environment Variables” → Edit the system environment variables → Environment Variables.

2. Under System variables, click New.

- Variable name: JAVA\_HOME

- Variable value: C:\Program Files\Eclipse Adoptium\jdk-17.0.18.8-hotspot (your Java folder path)
  - We then click Ok
3. Add Java to Path:
- In the same Environment Variables window, select the variable Path under System variables → Edit → New.
  - We add this to the line: %JAVA\_HOME%\bin  
We then click ok
4. To verify, we open the terminal and run `java -version` and `echo %JAVA_HOME%`

```
(taxic) C:\Users\rajve\Desktop\SkillIssues\Personal Projects\taxistream>java --version
openjdk 17.0.18 2026-01-20
OpenJDK Runtime Environment Temurin-17.0.18+8 (build 17.0.18+8)
OpenJDK 64-Bit Server VM Temurin-17.0.18+8 (build 17.0.18+8, mixed mode, sharing)

(taxic) C:\Users\rajve\Desktop\SkillIssues\Personal Projects\taxistream>echo %JAVA_HOME%
C:\Program Files\Eclipse Adoptium\jdk-17.0.18.8-hotspot
```

## STEP VIII: SPARK STREAMING

1. We now run this command in the terminal:  
`>python spark/stream_processor.py`
2. We also run this command, because even with Java installed, Spark still needs the Kafka connector.  
`>spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.8 spark/stream_processor.py`  
We will mostly be using this command.  
But, this will not work unless the following steps are completed too:
3. We go to the link <https://spark.apache.org/downloads.html>, and download Spark 3.5.8, Spark 3.5.8
4. A file gets downloaded: spark-3.5.8-bin-hadoop3
5. We Go to MY PC-> Local Disk C (C:\) ->Create a folder 'spark', and put the file 'spark-3.5.8-bin-hadoop3' inside it.
6. We now copy this file path
7. We go to Windows Search->Environment Variable, and under "System Variable", we make:
  - Variable name: SPARK\_HOME
  - Value: C:\spark\spark-3.5.8-bin-hadoop3
  - Click Ok
8. Under System Variable itself, we find "Path", we go there, and click Edit->New->and add: %SPARK\_HOME%\bin. Then: Ok Ok Ok.
9. In the terminal, we run command:  
`>set PYSPARK_PYTHON=C:\Users\rajve\anaconda3\envs\taxic\python.exe`  
-> This is needed because Spark doesn't automatically detect the Anaconda Python inside your **taxic** environment.
10. Now, in the Local C, we again create a folder: hadoop, and inside it, the folder: bin
11. We go to the link:  
<https://github.com/cdarlint/winutils/tree/master/hadoop-3.2.2/bin>
12. From there we download both winutils.exe and hadoop.dll.
13. Now we Set Environment Variables: We go to System Variable->New, and then add:

- Variable name: HADOOP\_HOME
- Value: C:\hadoop

14. In the System Variable itself, we find the “path” option, there we go to edit->new->and add:  
C:\hadoop\bin, or, %HADOOP\_HOME%\bin

#### STEP IX: TIME TO USE TWO TERMINALS TOGETHER

1. In Terminal 1, run:

```
>set PYSPARK_PYTHON=C:\Users\rajve\anaconda3\envs\taxic\python.exe
>spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.8
```

spark/stream\_processor.py

It will show as:

```

Batch: 0

+-----+-----+
|avg(fare_amount)|avg(trip_distance)|
+-----+-----+
| | |
+-----+-----+
```

But it won't start as of now.

2. In terminal 2:

```
>python producer/producer.py
```

As this goes on:

```
Sent ride 0
Sent ride 1
Sent ride 2
Sent ride 3
Sent ride 4
```

So does this too:

```
Batch: 1

+-----+-----+
|avg(fare_amount)|avg(trip_distance)|
+-----+-----+
| 10.0| 1.6|
+-----+-----+

Batch: 2

+-----+-----+
|avg(fare_amount)|avg(trip_distance)|
+-----+-----+
| 8.6| 1.06375|
+-----+-----+

Batch: 3

+-----+-----+
| avg(fare_amount)|avg(trip_distance)|
+-----+-----+
|10.053846153846154|1.3738461538461537|
+-----+-----+

Batch: 4

+-----+-----+
```

3. We can have a Quick Verification by going on the browser, and typing:

><http://localhost:4040>

-> This is the **Apache Spark UI**. There you can see:

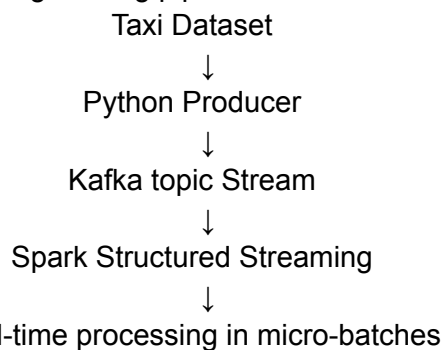
- Jobs
- Stages
- Streaming queries
- Execution DAG
- Processing times

->This is very useful when demonstrating the project.

| Job ID (Job Group)                        | Description                                                                                                                                 | Submitted           | Duration | Stages: Succeeded/Total | Tasks (for all stages): Succeeded/Total |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------|----------|-------------------------|-----------------------------------------|
| 11 (e37bcdef-a18f-4ded-82fc-46d24b874ac6) | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 11<br>start at NativeMethodAccessorImpl.java0 | 2026/03/15 02:03:57 | 1 s      | 2/2                     | 2/2                                     |
| 10 (e37bcdef-a18f-4ded-82fc-46d24b874ac6) | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 10<br>start at NativeMethodAccessorImpl.java0 | 2026/03/15 02:03:55 | 0.9 s    | 2/2                     | 2/2                                     |
| 9 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 9<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:03:52 | 1.0 s    | 2/2                     | 2/2                                     |
| 8 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 8<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:03:49 | 1 s      | 2/2                     | 2/2                                     |
| 7 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 7<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:03:47 | 1 s      | 2/2                     | 2/2                                     |
| 6 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 6<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:03:44 | 1 s      | 2/2                     | 2/2                                     |
| 5 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 5<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:03:42 | 1 s      | 2/2                     | 2/2                                     |
| 4 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 4<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:03:39 | 0.7 s    | 2/2                     | 2/2                                     |
| 3 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 3<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:03:37 | 1 s      | 2/2                     | 2/2                                     |
| 2 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 2<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:03:35 | 1 s      | 2/2                     | 2/2                                     |
| 1 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 1<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:03:31 | 2 s      | 2/2                     | 2/2                                     |
| 0 (e37bcdef-a18f-4ded-82fc-46d24b874ac6)  | id = add257f3-284b-4d73-9d5e-45cb3342c26 runId = e37bcdef-a18f-4ded-82fc-46d24b874ac6 batch = 0<br>start at NativeMethodAccessorImpl.java0  | 2026/03/15 02:02:57 | 3 s      | 1/1 (1 skipped)         | 1/1                                     |

## BRIEF OF WHAT HAS TO BE DONE NOW:

1. We now have a real-time data engineering pipeline:



Using:

- Apache Kafka
- Apache Spark
- Apache Spark Structured Streaming

2. Now, in short,

For Terminal 1:

```

>activate taxic
>cd "C:\Users\rjave\Desktop\SkillIssues\Personal Projects\taxistream"
>docker ps
>docker compose up -d
>set PYSPARK_PYTHON=C:\Users\rjave\anaconda3\envs\taxic\python.exe
>spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.8

```

spark/stream\_processor.py



And for Terminal 2:

>activate taxic

>cd "C:\Users\rajve\Desktop\SkillIssues\Personal Projects\taxistream"

>python producer/producer.py

3. Completed stages:

1. Environment setup
2. Spark installation
3. Hadoop utilities setup
4. Kafka services started with Docker
5. Producer script sending taxi ride events
6. Spark streaming job consuming Kafka data
7. Micro-batch processing working

The **end-to-end pipeline is functional**.

4. Till here, this project demonstrates a modern real-time data engineering pipeline that;

- simulates live data streams
- transports data via Kafka
- processes data using Spark streaming
- operates in micro-batch streaming mode

->It showcases the fundamental architecture used in large-scale data systems such as ride-sharing platforms, financial trading systems, and IoT analytics.

5. Skills that have been demonstrated till yet in this project:

- Streaming Data Engineering
- Kafka Event Pipelines
- Spark Structured Streaming
- Dockerized Infrastructure
- Real-time Data Processing
- Data Pipeline Architecture

---

### **BRIEF OF WHAT HAS TO BE DONE NOW:**

1. Data storage in apache Cassandra
  2. Workflow orchestration in Apache Airflow for scheduling pipelines.
  3. Create analytics dashboards using Power BI or Grafana
  4. Continuous streaming: To Modify the producer so it continuously streams events instead of stopping at 50 records.
-